Latent-Verbal Alignment: Characterizing SAE Feature Patterns Across CoT Reasoning Steps

Shreeti Shrestha
Northeastern University
Boston, MA 02115
shrestha.shre@northeastern.edu

Abstract

Chain-of-thought (CoT) reasoning is increasingly used as both a performance mechanism and an interpretability tool for monitoring AI behavior. Yet while recent work has begun probing step-level latent representations, direct measurement of semantic correspondence between active features and verbal content at each reasoning step remains largely absent. This paper introduces **Latent-Verbal Alignment (LVA)**, a correspondence score measuring the semantic match between auto-interpreted SAE feature labels and step-level reasoning text, providing a proxy for latent-semantic alignment. At each chain-of-thought step, LVA evaluates the top-k most active SAE features against the corresponding verbal content. Using Goodfire's pre-trained sparse autoencoder on DeepSeek R1-671B, 5,994 (step, feature) pairs are scored across 93 mathematical reasoning traces from OpenR1-Math-220k. Alignment is sparse but precise: 88.5% of activated features are broad context encoders with no step-specific correspondence, while the remaining 11.5% are genuinely step-local. Alignment varies across reasoning stage types, but the driver is encoding frequency rather than correspondence quality. Self-correction steps are least likely to activate a step-specific feature (28.6% vs. 41.2% for setup/planning), yet when they do, they show the highest conditional match precision (mean LVA=0.652), a directional reversal that warrants replication at larger scale. Analysis of persistently low-scoring features reveals that auto-interpreted SAE labels can misfire in two ways: syntactic features mislabeled as semantic, and domain-specific labels applied to broadly-activating features. Together, these findings provide a methodological prototype for characterizing layer-level representational structure during CoT reasoning, and suggest that even at a semantically rich middle layer, generic contextual encoding predominates over step-specific computation. [1]

1 Introduction

The ability to monitor the internal reasoning of large language models (LLMs) is a central goal of AI interpretability research. Chain-of-thought (CoT) prompting [1, 2], a technique that elicits step-by-step reasoning by instructing models to verbalize intermediate steps before producing a final answer, has emerged as a promising candidate for this role. By making intermediate reasoning explicit, it appears to offer a window into model computation that is both human-readable and mechanistically grounded. This dual role as a performance enhancer and interpretability tool has made CoT central to proposals for AI safety monitoring, where the assumption is that what a model writes reflects what it computes.

That assumption is increasingly under pressure. Turpin et al. [3] demonstrated that inserting biasing features into prompts changes model answers in a large fraction of cases, yet the CoT trace almost

[1] Implementation code for the experiments is available at: https://github.com/shreetishresthanp/latent_verbal_gap

never mentions these influences. Chen et al. [4] extended this finding to DeepSeek-R1 and Claude 3.7 Sonnet, showing systematic omission of hidden metadata hints even in state-of-the-art reasoning models. Zaman and Srivastava [5] offer a rebuttal, arguing that this metric conflates unfaithfulness with incompleteness and that non-verbalized hints can still causally mediate prediction changes through the CoT. Regardless of how this debate resolves, all of these approaches operate at the text level: they identify discrepancies between prompt content and verbalized reasoning without examining the internal representations driving each step. Without directly measuring step-level correspondence between active features and verbal content, the reliability of CoT as a monitoring tool cannot be properly evaluated.

Sparse autoencoders (SAEs) offer a path toward closing this gap. By decomposing high-dimensional residual stream activations into sparse, interpretable features [6, 7], SAEs provide step-level access to the internal representations of a model during generation. Recent work has shown that SAE features can causally influence CoT outputs [8] and that step-level latent vectors encode recognizable reasoning behaviors [9]. Yet these approaches either aggregate across the full trace or focus on final-token representations, stopping short of directly measuring whether active features correspond to what the model verbalizes at each step. This is the precise question that CoT monitoring implicitly assumes is answered in the affirmative: do SAE features most active at each CoT reasoning step semantically correspond to the verbal content of that step?

This paper introduces Latent-Verbal Alignment (LVA), a quantitative score measuring the semantic correspondence between the top-k active SAE features at each CoT step and that step's verbal content. Goodfire recently open-sourced two SAEs trained on DeepSeek R1-671B [10], the first publicly available interpreter models trained on a true reasoning model at this scale. This work uses the layer 37 SAE, which contains 28,672 features with auto-interpreted labels generated by Claude 3.7 Sonnet. LVA scores are computed for 5,994 (step, feature) pairs across 93 mathematical reasoning traces from OpenR1-Math-220k, using an LLM-as-judge scoring pipeline with demonstrated high reliability. Three hypotheses are evaluated:

- whether alignment exists above chance baselines (H1)
- whether it varies systematically across reasoning stage types (H2), and
- whether a subset of features is persistently misaligned with any verbal content (H3)

Figure 1 illustrates the full pipeline.

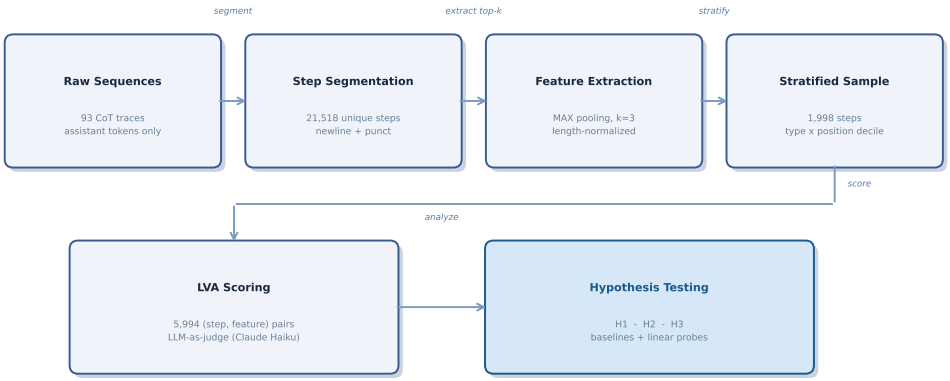


Figure 1: LVA pipeline overview. Raw CoT traces are segmented into steps, top-k SAE features are extracted per step using `MAX` pooling with length normalization. After identifying step category and sampling from each category, each step is paired with its top-3 features and scored by an LLM-as-judge, yielding 5,994 (step, feature) pairs for hypothesis testing.

Summary of contributions. The contributions of this work are as follows:

- A step-level correspondence measurement framework, LVA, introduced and validated on activations from state-of-the-art reasoning model.
- A three-baseline design that disambiguates genuine step-local alignment from generic mathematical overlap.
- An empirical characterization of layer 37's representational structure: alignment is sparse but precise, with the layer functioning predominantly as a context encoder rather than a step-specific computation tracker.
- Evidence that the structure of latent-verbal alignment varies by reasoning stage: self-correction steps show the lowest encoding rate but highest conditional correspondence quality, suggesting the latent-verbal gap is more nuanced than a simple faithfulness deficit.
- A case study demonstrating that SAE auto-interp labels can systematically misrepresent feature function, motivating label quality checks in LVA-style measurement pipelines.

Section 2 situates this work within the related literature. Section 3 describes the methodology, including trace segmentation, feature extraction, and the LVA scoring pipeline. Section 4 presents results and analysis for each hypothesis. Section 5 discusses limitations and directions for future work with Section 6 discussing the broader impact of this work.

2 Related Works

CoT faithfulness and monitoring. Chain-of-thought reasoning has been proposed as an interpretability tool on the premise that verbalized reasoning reflects internal computation. Turpin et al. [3] challenged this premise by showing that biasing features inserted into prompts influence model answers without appearing in the CoT trace. Chen et al. [4] extended this to state-of-the-art reasoning models, demonstrating systematic omission of hidden metadata hints in DeepSeek-R1 and Claude 3.7 Sonnet. Zaman and Srivastava [5] offer a partial rebuttal, arguing that non-verbalized hints can still causally mediate prediction changes through the CoT, and that apparent unfaithfulness often reflects lossy compression rather than deception. Lanham et al. [11] provide complementary empirical evidence, showing via truncation experiments that CoT steps are not always necessary for correct answers. Regardless of how this debate resolves, all of these approaches operate at the text level without examining the internal representations driving each step, leaving open the question of what the model is actually computing when it writes each reasoning step.

Sparse autoencoders for interpretability. Mechanistic interpretability offers a path toward answering this question by examining model internals directly. Sparse autoencoders address the polysemanticity problem in neural networks [6, 7], where individual neurons respond to multiple unrelated concepts, by learning a sparse overcomplete dictionary of features from residual stream activations. The resulting features are more interpretable and monosemantic, and crucially, causally manipulable: activation patching experiments have demonstrated that steering individual SAE features produces targeted changes in model output [6]. Auto-interpreted labels, generated by prompting an LLM with maximally activating examples, provide natural language descriptions of each feature and have become a standard tool for large-scale SAE analysis [7]. However, SAEs alone do not answer whether the features active at a given moment correspond to what the model verbalizes, which is why recent work has focused on using it in conjunction with CoT reasoning.

SAEs applied to CoT reasoning. Recent work has begun applying SAEs directly to reasoning model activations. Chen et al. [8] train separate SAEs on CoT and no-CoT activations of smaller models, using activation patching to show that CoT-specific features causally boost correct answer probability, but this analysis operates only at the final token rather than at each reasoning step. Zhang et al. [9] train SAEs on step-level activations of DeepSeek-R1-1.5B, discovering disentangled latent vectors for reflection, backtracking, and confidence that can modulate reasoning trajectories at inference time, but never measure whether these vectors correspond to what the model writes at each step. Hazra et al. [12] identify reasoning-related SAE features via an introspective vocabulary metric and provide causal evidence through steering experiments, but similarly do not measure step-level verbal correspondence. These works collectively establish that SAE features encode meaningful reasoning structure, but stop short of directly measuring whether active features correspond to what the model verbalizes at each step; the precise question this work addresses.

3 Methods

3.1 Data and Infrastructure

Goodfire has open-sourced two sparse autoencoders trained on DeepSeek R1-671B [10], the first publicly available SAEs trained on a true reasoning model at a large scale. This work uses the layer 37 SAE, which contains 28,672 features with auto-interpreted labels generated by Claude 3.7 Sonnet. Feature activations are pre-computed and stored in a DuckDB database covering the open-r1/OpenR1-Math-220k distribution [13]. Only sequences with confirmed activation records in the pre-computed database are used. Querying 0.1% of the database for distinct sequence IDs with activation records yielded 93 usable sequences, each containing a full chain-of-thought trace. All experiments use a fixed random seed for reproducibility.

3.2 Trace Segmentation and Feature Extraction

Each CoT trace is segmented into steps using a boundary detection heuristic applied to assistant tokens only, identified by the `<|Assistant|>` delimiter. A new step boundary is marked when two consecutive tokens both contain newline characters (`\n`), or when the current token ends with sentence-final punctuation (`.`, `!`, or `?`), excluding cases where a period follows a digit to avoid splitting numerical expressions such as `3.14`. Special tokens including `<think>`, `</think>`, and role delimiters are stripped before text processing. Steps shorter than 20 characters after cleaning are discarded as uninformative fragments. This yields 21,518 unique steps across 93 sequences, with a mean of 231 steps per sequence and a range of 8 to 798.

For each step, the top-k SAE features are extracted using `MAX` pooling over the token span: for each feature active anywhere within the step, the maximum activation strength across all tokens in that span is recorded. Because raw activation strength correlates with step length (Spearman $r = 0.52$), each feature's MAX activation is divided by token length and normalized to sum to one across the top-k features; this reduces the correlation to $r \approx 0$.

The value of k is set to 3 for all reported results. Inspection of activation strength distributions at $k \in \{5, 10\}$ showed that features ranked 4 and beyond had near-identical activation strengths, suggesting they encode the same generic mathematical context rather than step-specific content. Using $k = 3$ retains the most informative features while avoiding noise from lower-ranked generic activations.

Step type is assigned using keyword heuristics applied to the step text, inspired by the approach of Zhang et al. [9], classifying each step into one of five categories: *calculation*, *setup/planning*, *self-correction*, *uncertainty*, and *other*. Table 1 summarizes the keyword patterns used for each category.

Step Type	Identification Criteria
Calculation	Arithmetic operators, LaTeX math expressions (e.g. `\frac`, `\sqrt`), and computation verbs (`compute`, `simplify`, `expand`)
Setup/planning	Planning phrases (`let's`, `we need to`, `the problem asks`, `approach`, `let me define`)
Self-correction	Metacognitive markers (`wait`, `actually`, `hold on`, `let me reconsider`, `I was wrong`)
Uncertainty	Hedging language (`perhaps`, `not sure`, `might be`, `could be`, `I'm not certain`)
Other	No pattern matched

Table 1: Step type classification criteria. Patterns are applied via case-insensitive regex matching; the first matching category is assigned.

To make LVA scoring computationally feasible, a stratified sample of 1,998 steps is drawn from the full 21,518, stratified by step type and relative position decile. Table 2 shows the resulting composition.

Step Type	Count	% of sample
Calculation	574	28.7%
Other	939	47.0%
Setup/planning	170	8.5%
Self-correction	168	8.4%
Uncertainty	147	7.4%

Table 2: Stratified sample composition by step type (1,998 steps total). Each step is paired with its top-3 features, yielding 5,994 (step, feature) pairs for scoring.

3.3 LVA Correspondence Scoring

Each `(step, feature)` pair is scored by an LLM-as-judge pipeline using Claude Haiku (`claude-haiku-4-5`). The judge receives the step text and the feature's auto-interpreted label, and rates their semantic correspondence on a 0–1 scale. The scoring rubric distinguishes five bands, shown in Table 3. The judge is explicitly instructed to be strict: a generic label such as `"mathematical reasoning"` should not score highly simply because both the label and the step involve mathematics. The full system prompt is provided in Appendix A. The judge returns a `JSON` object with three fields: `"score"` (float), `"confidence"` (`"high"`, `"medium"`, or `"low"`), and `"reasoning"` (one sentence string).

Score Range	Interpretation
0.0–0.2	No relation. Feature describes something entirely absent from this step.
0.3–0.4	Generic overlap only. Label technically applies but is not specifically triggered by this step.
0.5–0.6	Relevant but imprecise. Label is related but not specific to this step's operation.
0.7–0.8	Strong match. Label clearly describes something present in this step.
0.9–1.0	Precise match. Label describes the dominant operation in this step.

Table 3: LLM-as-judge scoring rubric for LVA correspondence scoring. Scores of ≤ 0.40 are treated as generic; scores > 0.40 as specific.

Scoring is parallelized across `max_workers=3` threads with exponential backoff on failures and checkpointed every `CHECKPOINT_EVERY=200` rows to `.parquet` format. All 5,994 submitted pairs returned valid scores with zero `"error"` confidence responses.

Judge reliability is assessed by scoring a 1% subsample ($n = 60$ pairs) twice in independent runs and computing the inter-run Pearson correlation. The resulting $r = 0.980$ ($p < 0.001$) confirms high scorer consistency, well above the $r \geq 0.80$ threshold set as the reliability criterion prior to the full scoring run.

The per-step LVA score is computed as the strength-weighted mean of the k feature scores:

$$\text{LVA}(s) = \sum_{i=1}^{k} w_i \cdot \text{score}_i \tag{1}$$

where w_i is the length-normalized activation weight (`strength_w`) of feature i at step s, and $\text{score}_i \in [0, 1]$ is the LLM judge score (`lva_score`) for the corresponding `(step, feature)` pair. A feature is considered *specific* if $\text{score}_i > 0.40$, and *generic* otherwise. A step is considered to have activated a specific feature if at least one of its top-$k = 3$ features exceeds this threshold.

3.4 Hypothesis Testing

H1: Baseline Comparison. Three baselines test whether LVA scores above the generic ceiling (> 0.40) reflect genuine step-specific correspondence rather than an artifact of the scoring pipeline or the mathematical domain:

- *Random-feature baseline*: Globally permutes `lva_score` values across all `(step, feature)` pairs, testing whether specific features are more informative than arbitrary ones drawn from the same distribution.
- *Step-shuffle baseline*: Shifts step indices forward by one position within each sequence (`step_idx` - i paired with text of `step_idx` - $i + 1$, wrapping at sequence boundaries), testing whether features are specifically aligned to the step they activate on rather than any adjacent step.
- *Cross-problem baseline*: Substitutes same-position steps across different sequences by shuffling `sequence_id` within relative position deciles, controlling for structural position effects.

Statistical significance is assessed via two-sided Welch's t-test; Cohen's d is reported as effect size, with $d > 0.3$ as the threshold for practical significance.

H2: Reasoning Stage Analysis. Two analyses examine whether alignment varies across step types.

- *Rate analysis*: Computes the proportion of steps where at least one specific feature activates (`max_lva` > 0.40 across top-k features) for each `step_type`, across all 1,998 sampled steps. A χ^2 contingency test assesses whether specific-feature rates differ across step types, followed by a planned z-test comparing `self_correction` against `setup_planning` as the highest-rate category.
- *Conditional quality analysis*: Restricts to steps where a specific feature activated and tests whether mean `lva_score` differs across step types using one-way ANOVA (F-test). This addresses whether, conditional on a specific feature being present, correspondence quality varies by step type.

As an independent validation, two linear probes are trained on the raw SAE feature activation matrix (n steps $\times$ 1,498 features, using raw `strength` values). The dataset is split 80/20 by `sequence_id`. **Probe A** (`LogisticRegression`) predicts `step_type` from activations; macro F1 is reported against a permutation baseline (100 shuffles). **Probe B** (`RidgeRegression`) predicts `lva_score` from activations; R^2 is reported against a mean-prediction baseline. Both probes test whether SAE activation patterns encode step-level information independently of the correspondence scoring pipeline.

H3: Silent Feature Analysis. This examines features that are active but semantically invisible, i.e. they fire strongly at steps but their autointerp label doesn't correspond to what the step is doing. Feature-level mean `lva_score` is computed across all scored pairs, grouping by `feature_id`. Features with fewer than two observations are excluded, yielding 81 reliable features. The bottom 5th percentile of the mean `lva_score` distribution (threshold ≤ 0.12) defines the silent feature set, producing 5 candidate features. Given the small number of candidates relative to the target of 15–20, a population-level claim is not supported.

Instead, two features are selected for qualitative case study analysis: Feature `#23364` (*Commas in mathematical reasoning*) scores low not because it activates broadly on meaningful content, but because punctuation has nothing semantically useful to say about any reasoning step. Feature `#3690` (*Mathematical reasoning with prime numbers and divisibility tests*) is identified via a supplementary analysis relaxing the minimum observation threshold to $n \geq 10 (n = 228)$ observations, mean `lva_score` $= 0.154$). This feature depicts the core silent feature failure: it activates broadly across mathematical context yet carries a misleadingly specific autointerp label, making it genuinely active but semantically misrepresented.

4 Results

4.1 H1: Alignment is Sparse but Precise

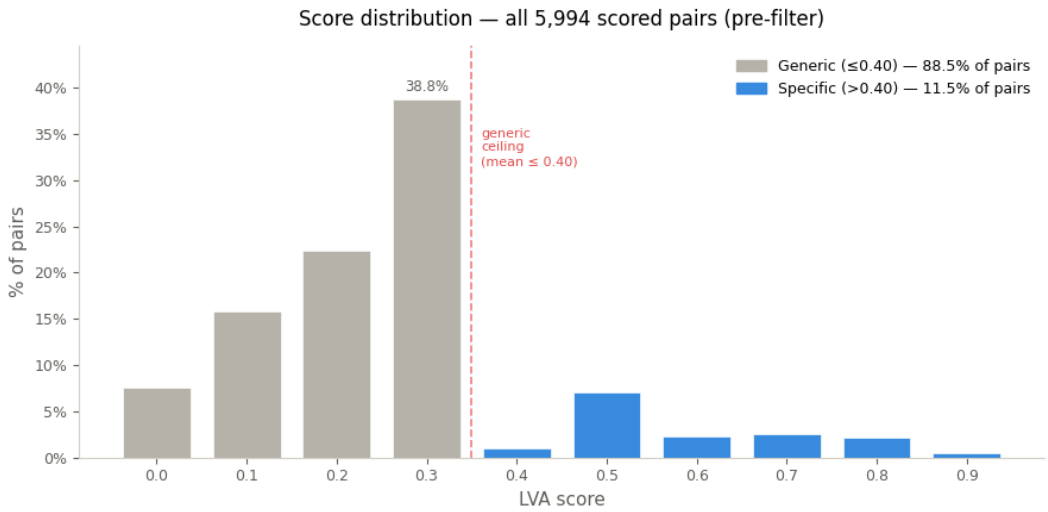


Figure 2: LVA score distribution across all 5,994 valid scored pairs. 88.5% of pairs score at or below the generic ceiling (≤ 0.40), with the modal bin at 0.3–0.4 (38.8%). Only 11.5% are step-specific (> 0.40).

Figure 2 shows the distribution of all 5,994 valid LVA scores. The distribution is strongly left-skewed: 88.5% of pairs score at or below the generic ceiling (≤ 0.40), with the modal bin at 0.3–0.4 (38.8%). Only 11.5% of pairs are step-specific, suggesting layer 37 functions predominantly as a broad context encoder. To isolate genuinely specific features, a label fidelity filter is applied: features with per-feature mean `lva_score` ≤ 0.40 across $n \geq 5$ scored pairs are classified as generic. This yields 651 specific-feature steps used in subsequent analyses.

Table 4 reports baseline results:

Baseline	Mean LVA	Baseline Mean	Cohen's d	p
Random-feature	0.592	0.598	-0.023	0.682
Cross-problem	0.592	0.626	-0.135	0.453
Step-shuffle	0.592	0.340	1.058	< 0.001

Table 4: H1 baseline comparison. Mean LVA is the mean score of specific features on their true steps. Baseline Mean is the mean score under each counterfactual condition. Step-shuffle is the only significant baseline ($d = 1.058$), confirming step-local specificity.

- *Random-feature* ($d = -0.023$, $p = 0.682$): null. Specific features are not more informative than arbitrary ones, confirming labels are genuinely descriptive.
- *Cross-problem* ($d = -0.135$, $p = 0.453$): null (but underpowered).Same-position steps across problems share structural similarity, masking potential position effects.
- *Step-shuffle* ($d = 1.058$, $p < 0.001$): significant. Mean LVA drops from 0.592 to 0.340 when features are paired with adjacent step text, confirming that specific features are genuinely step-local. Table 5 illustrates this two-part structure with a representative example.

4.2 H2: Alignment Varies by Encoding Frequency

Rate Analysis. Figure 3 shows specific-feature rates by step type. Setup/planning has the highest rate (41.2%) and self-correction the lowest (28.6%): when writing a self-correction step, none of

Rank	Type	Feature Label	LVA
1	generic	Mathematical reasoning and step-by-step problem solving	0.10
2	generic	Mathematical problem-solving with formal notation	0.10
3	**specific**	**Self-reference to previous reasoning steps**	**0.95**
Rank-3 feature paired with adjacent step			0.10

Table 5: Top-3 active features for step *"Wait, we already did that"* (sequence 73000, step 157). Rank-1 and rank-2 are generic context encoders; rank-3 is step-specific. Pairing the rank-3 feature with the adjacent step drops the score from 0.95 to 0.10.

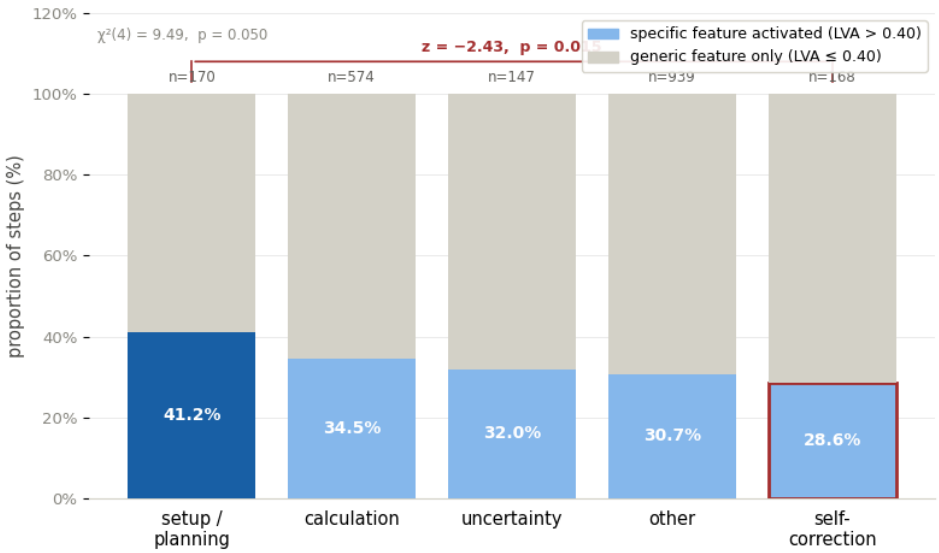


Figure 3: Specific-feature rate by step type across all 1,998 sampled steps. Setup/planning has the highest rate (41.2%) and self-correction the lowest (28.6%). $\chi^2(4) = 9.49$, $p = 0.050$; planned z-test: $z = -2.43$, $p = 0.015$.

the top-3 active features specifically describe what the model is doing 71.4% of the time. The χ^2 omnibus test is marginally significant ($\chi^2(4) = 9.49$, $p = 0.050$); a planned z-test confirms the contrast ($z = -2.43$, $p = 0.015$).

Conditional Quality Analysis. Among the 651 specific-feature steps identified in H1, one-way ANOVA yields a marginally significant omnibus result `lva_score` ($F(4, 646) = 2.49$, $p = 0.042$), but no pairwise contrast survives Bonferroni correction across the ten pairs tested. The point estimates show a directional reversal: self-correction has the highest conditional mean LVA (0.652) despite having the lowest encoding rate, while setup/planning has the lowest conditional mean (0.587) despite the highest encoding rate. This reversal is consistent with the interpretation that metacognitive language, when it does activate a specific feature, produces more distinctive correspondence, but the self-correction cell contains only n=48 steps. The ANOVA result is therefore treated as a pattern worth investigating rather than a confirmed effect: the omnibus p-value clears the nominal threshold, but the small cell size and absence of surviving pairwise contrasts mean the ordering could reflect sampling variability. Table 6 reports descriptive statistics (see Appendix 7 for the full distribution).

Linear Probe. Probe A (`LogisticRegression`) achieves macro F1 $= 0.321$ against a permutation baseline of 0.148 ± 0.011, corroborating H2 from raw activations alone. Per-class recall mirrors the rate analysis: `calculation` (0.52) and `other` (0.79) are most recoverable; `self_correction`,

Step Type	Mean LVA	SD	n
Self-correction	0.652	0.227	48
Calculation	0.622	0.202	198
Uncertainty	0.593	0.276	47
Setup/planning	0.587	0.251	70
Other	0.563	0.262	288

Table 6: Conditional quality analysis: mean `lva_score` by step type among 651 specific-feature steps. One-way ANOVA: $F(4, 646) = 2.49$, $p = 0.042$; no pairwise contrast survives Bonferroni correction ($n = 10$ pairs). Self-correction point estimate (0.652) is highest but based on n=48 steps; direction is suggestive, not confirmatory.

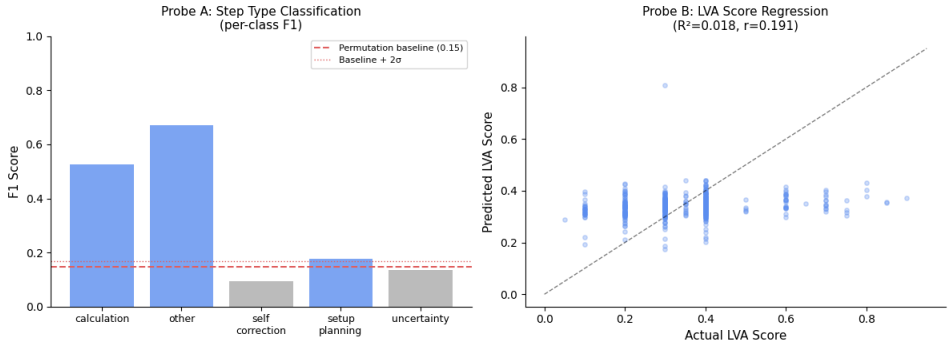


Figure 4: Linear probe results. **Left (Probe A):** per-class F1 for `LogisticRegression` predicting `step_type` from SAE activations. Calculation and other exceed the permutation baseline (0.15); self-correction, setup-planning, and uncertainty are near noise. **Right (Probe B):** predicted vs. actual `lva_score` from `Ridge` regression ($R^2 = 0.018$), indicating activations carry minimal signal for predicting continuous LVA scores.

`uncertainty`, and `setup_planning` are near noise (0.05–0.10). Probe B (`Ridge`) achieves $R^2 = 0.018$, indicating activations carry minimal signal for predicting continuous LVA scores.

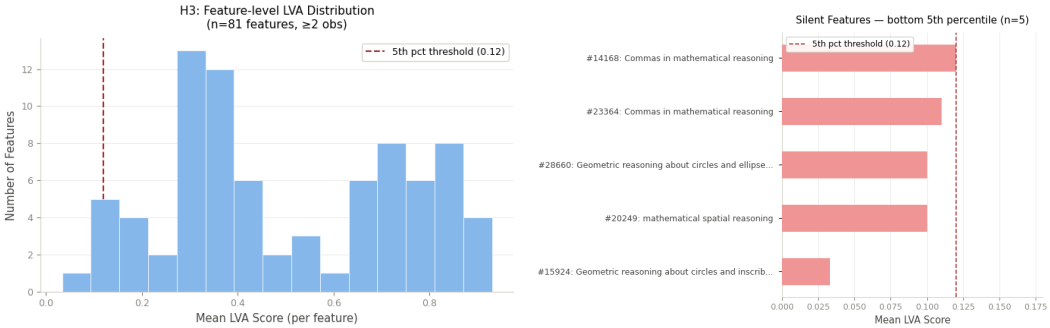
4.3 H3: Silent Features Reveal Label Fidelity Failures

Feature-level mean `lva_score` is computed for all features with $n \geq 2$ observations, yielding 81 reliable features. The bottom 5th percentile (threshold ≤ 0.12) produces 5 silent feature candidates (Figure 5), well below the target of 15–20; a population-level claim is not supported. Two representative cases are examined.

Syntactic Features. Feature `#23364` (*Commas in mathematical reasoning*, $n = 10$, mean LVA $= 0.110$) activates highly across 43 sequences but scores low in alignment: *"Wait, but x can be 0?"* (score 0.0), *"Wait, that's too big."* (score 0.0). The label describes a syntactic property but the feature activates broadly on mathematical prose. This feature also appears as the strongest positive predictor of `self_correction` in Probe A (coef $= 2.050$), likely contaminating that signal.

Over-specific Labels. Feature `#15924` (*Geometric reasoning about circles and inscribed triangles*, $n = 3$, mean LVA $= 0.033$) activates on steps with no geometric content: *"So no solution here."* (score 0.0). The label overstates feature specificity.

Supplementary Analysis. Relaxing to $n \geq 10$ observations identifies Feature `#3690` (*Mathematical reasoning with prime numbers and divisibility tests*, $n = 228$, mean LVA $= 0.154$) as the most substantive case (Figure 6). Despite high activation frequency across all 93 sequences, this suggests the feature functions as a broad mathematical background activator rather than a selective domain detector.



(a) Feature-level LVA distribution (n=81 features, ≥ 2 obs). Bottom 5th percentile threshold $= 0.12$.

(b) Five silent feature candidates below the 5th percentile threshold.

Figure 5: H3: Feature-level LVA distribution and silent feature candidates. Five features fall below the 5th percentile threshold (≤ 0.12), with the lowest feature LVA score $= 0.033$).

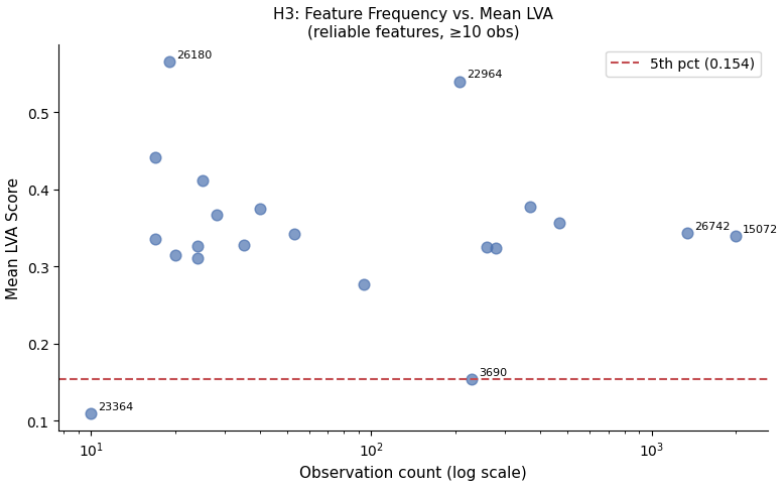


Figure 6: Feature frequency vs. mean LVA score among reliable features (≥ 10 obs). Feature #3690 is anomalous: high observation count ($n = 228$) but mean LVA $= 0.154$, below the 5th percentile threshold.

These cases illustrate two label fidelity failure modes: syntactic features mislabeled as semantic, and domain-specific labels applied to broadly-activating features.

4.4 Sanity Checks

Judge Consistency. A 1% subsample ($n = 60$ pairs) was scored twice in independent runs. The inter-run Pearson correlation $r = 0.980$ ($p < 0.001$) confirms high scorer reliability, well above the pre-specified threshold of $r \geq 0.80$.

Length Normalization. Raw activation strength correlates with step token length (Spearman $r = 0.52$). After dividing by step length and renormalizing, the correlation drops to $r \approx 0$, confirming that `strength_w` captures activation intensity independently of step length.

5 Limitations and Future Work

Auto-interp Label Quality. LVA scores are computed by comparing auto-interpreted labels against step text, creating a ceiling on correspondence scores if labels are too generic or inaccurate. The H3

findings directly demonstrate this problem. Future work should explore richer feature descriptions using maximally activating exemplars rather than single auto-interp labels.

Scale and Statistical Power. Only 93 sequences were available from the pre-computed activation database, limiting power for H2 (self-correction $n = 48$, bootstrap check not completed) and H3 (5 silent candidates against a target of 15–20). Scaling to the full 1% activation sample (47GB) is the most direct path to replicating the H2 reversal and identifying a robust silent feature population.

Single Layer and Domain. All analyses are conducted on layer 37 of R1-671B using mathematical reasoning traces only. Extending to multiple layers and non-mathematical domains would test whether the 88.5% generic finding reflects a property of this layer or a broader characteristic of R1's representational structure. Causal validation via activation patching on smaller distilled models remains the most important longer-term direction.

Step Type Classification. Keyword heuristics, following [9], may undercount self-correction steps expressed through subtler language, inflating the *other* category and affecting H2 comparisons. The rank ordering finding (rank-1 mean LVA $= 0.337$, rank-3 mean LVA $= 0.396$) further suggests the top-k design may miss alignment signal in lower-ranked features. Both issues motivate LLM-based step classification and broader feature selection strategies in future work.

6 Discussion and Conclusion

The central finding, that 88.5% of active SAE features are generic context encoders, suggests that a single-layer CoT monitor would capture contextual framing reliably but miss step-specific computation most of the time. The 11.5% that are step-specific are precisely step-local (step-shuffle $d = 1.058$). The H2 reversal is the most mechanistically interesting result: self-correction steps are least likely to activate a specific feature (28.6%), yet when they do, the match is most precise (mean LVA $= 0.652$). If this pattern replicates at larger scale, it would suggest that metacognitive language occupies a sparse but distinctive region of layer 37's feature space: encoded rarely, but precisely when encoded. H3 reframes silent features as label fidelity failures rather than hidden computation.

This work introduces LVA as a prototype for characterizing layer-level representational structure during CoT reasoning. If CoT monitoring is to serve as a safety mechanism, it must catch the computations that actually determine model behavior. This work highlights the importance of identifying structure and magnitude of feature-reasoning alignment. Because LVA evaluates correspondence using auto-interpreted feature labels, it serves as a proxy for latent-semantic alignment and is inherently constrained by label quality. Scaling to larger samples, extending to non-mathematical domains, and causal validation via activation patching are the most productive next steps.

References

[1] Jason Wei et al. "Chain-of-thought prompting elicits reasoning in large language models". In: *Advances in neural information processing systems* 35 (2022), pp. 24824–24837.

[2] Takeshi Kojima et al. "Large language models are zero-shot reasoners". In: *Advances in neural information processing systems* 35 (2022), pp. 22199–22213.

[3] Miles Turpin et al. "Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting". In: *Advances in Neural Information Processing Systems* 36 (2023), pp. 74952–74965.

[4] Yanda Chen et al. "Reasoning models don't always say what they think". In: *arXiv preprint arXiv:2505.05410* (2025).

[5] Kerem Zaman and Shashank Srivastava. "Is Chain-of-Thought Really Not Explainability? Chain-of-Thought Can Be Faithful without Hint Verbalization". In: *arXiv preprint arXiv:2512.23032* (2025).

[6] Hoagy Cunningham et al. "Sparse autoencoders find highly interpretable features in language models". In: *arXiv preprint arXiv:2309.08600* (2023).

[7] Trenton Bricken et al. "Towards monosemanticity: Decomposing language models with dictionary learning". In: *Transformer Circuits Thread* 2.5 (2023), p. 6.

[8] Xi Chen, Aske Plaat, and Niki van Stein. "How does chain of thought think? mechanistic interpretability of chain-of-thought reasoning with sparse autoencoding". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 40. 36. 2026, pp. 30297–30305.

[9] Zhenyu Zhang et al. "Fantastic Reasoning Behaviors and Where to Find Them: Unsupervised Discovery of the Reasoning Process". In: *arXiv preprint arXiv:2512.23988* (2025).

[10] Dron Hazra et al. "Under the hood of a reasoning model". In: *Goodfire Research Blog, April* (2025).

[11] Tamera Lanham et al. "Measuring faithfulness in chain-of-thought reasoning". In: *arXiv preprint arXiv:2307.13702* (2023).

[12] Andrey Galichin et al. *I Have Covered All the Bases Here: Interpreting Reasoning Features in Large Language Models via Sparse Autoencoders*. 2025. arXiv: `2503.18878 [cs.CL]`. URL: `https://arxiv.org/abs/2503.18878`.

[13] Hugging Face. *open-r1/OpenR1-Math-220k*. Accessed: [Insert Date Here]. 2025. URL: `https://huggingface.co/datasets/open-r1/OpenR1-Math-220k`.

A LLM-as-Judge System Prompt

The following system prompt is used for all LVA correspondence scoring calls to `claude-haiku-4-5`:

```
You are evaluating whether a SAE feature label semantically
corresponds to a reasoning step from a math chain-of-thought trace.

"Corresponds" means: given this step's content, would you expect
this feature to be active? Be strict - a generic label like
"mathematical reasoning" should NOT score highly just because
both involve math.

Score guide:
0.0-0.2  No relation. Feature describes something entirely absent.
0.3-0.4  Generic overlap only. Label technically applies but isn't
         specifically triggered by this step.
0.5-0.6  Relevant but imprecise. Label is related, not specific.
0.7-0.8  Strong match. Label clearly describes something in this step.
0.9-1.0  Precise match. Label describes the dominant operation here.

Confidence:
"high"   - correspondence is unambiguous
"medium" - step or label is somewhat ambiguous
"low"    - truncated step, very abstract label, or genuine uncertainty

Return only JSON: {"score": float, "confidence": str, "reasoning": str}
One sentence max for reasoning.
```

B Supplementary Figures

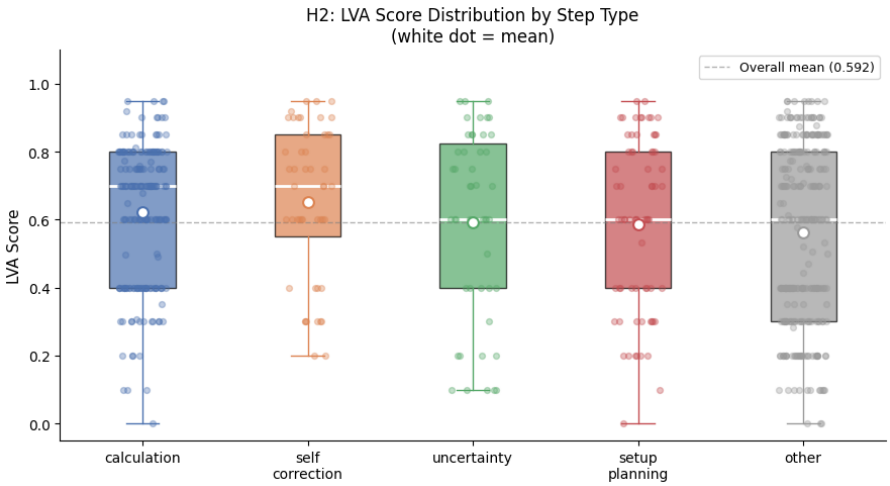


Figure 7: LVA score distribution by step type among 651 specific-feature steps (white dot = mean, dashed line = overall mean 0.592). Self-correction shows the highest mean (0.652) with wide variance consistent with the small sample ($n = 48$).

C Code Availability

Implementation code is available at: `https://github.com/shreetishresthanp/latent_verbal_gap.git`

Use of LLMs: LLM has been used to generate parts of code. While LLM was used for polish and formatting in report writing, the critiques, and rationale for design are my own.